université de BORDEAUX

Automatic tuning of the IO500 benchmark parameters with Bayesian Optimization

M2 Computer Science
Internship Report

Inria Centre at the University of Bordeaux

[TODO: student name**]**
Supervisors: **[TODO:** supervisor names**]**

August 13, 2026

Contents

1. Introduction

- **Context: I/O performance and the IO500 benchmark.**
  - HPC applications are increasingly limited by I/O rather than by compute: data volumes grow faster than the storage system's ability to absorb them.

    High Performance Computing (HPC) systems are used to run large-scale scientific and engineering applications. These applications often process very large amounts of data and use many computing resources in parallel. As a result, their performance does not only depend on the computing power of the system. The ability to access and store data efficiently is also an important factor. In many HPC workloads, I/O operations can become a bottleneck and limit the overall application performance.

    This problem is becoming more important as the amount of data produced and processed by HPC applications continues to increase. The growth of data can be faster than the ability of storage systems to absorb and process it. Improving I/O performance is therefore an important research and engineering problem in HPC.

  - Parallel file systems (Lustre, BeeGFS, GPFS) expose many tunable knobs (striping, number of storage targets, transfer size, process placement), and the right values are machine dependent.

    Parallel file systems such as (Lustre, BeeGFS, GPFS) are commonly used to provide high-performance storage in HPC environments. These systems expose several configuration parameters that can affect I/O performance. For example, performance can depend on the striping configuration, the number of storage targets used, the transfer size, the number of processes, or the placement of processes on compute nodes. The best values for these parameters are not necessarily the same for every machine. A configuration that performs well on one system may perform poorly on another system.

    This makes the evaluation of I/O performance difficult. A benchmark must not only measure the performance of the storage system, but should also provide a meaningful way to compare different systems and configurations.

  - IO500 is the de-facto standard benchmark for ranking HPC storage systems: it combines bandwidth-oriented and metadata-oriented phases into a single score, and publishes a biannual list.

    The IO500 benchmark suite was designed for this purpose. IO500 is widely used to evaluate and rank the I/O performance of HPC storage systems. It combines several workloads, including bandwidth-oriented and metadata-oriented operations, into a single score. The benchmark results are also used to produce the IO500 ranking, which is published twice a year.

  - Because IO500 is a *ranking*, sites have a direct incentive to tune it; the published score of a system therefore depends as much on how it was configured for the run as on the hardware itself.

    Because IO500 is a ranking, there is a direct incentive for HPC sites to obtain the best

possible score. The reported performance of a system therefore does not only depend on its hardware. It also depends on how the system is configured and how the benchmark is executed. Finding a good configuration can consequently have a significant impact on the measured IO500 score.

- **Problem: proper tuning is out of reach.**
  - The IO500 submission rules require every write/create phase to run for at least 300 s (stonewall timer set to 300 s); a run whose write phase finishes earlier is marked **invalid**.

    The large number of possible configurations makes the tuning of IO500 difficult. A configuration can involve several parameters, and each parameter can have multiple possible values. The combination of these parameters creates a potentially large search space. Finding a good configuration therefore requires evaluating many possible combinations.

    Another important difficulty is the cost of an IO500 evaluation. According to the IO500 submission rules, each write/create phase must run for at least 300 seconds, using a stonewall timer set to 300 seconds. A run that finishes the write phase earlier is considered invalid. This requirement makes a valid IO500 measurement relatively expensive in terms of cluster time.
  - Consequence: one *valid* IO500 measurement costs roughly 30 min of exclusive cluster time.

    On the target platform, one valid IO500 measurement takes approximately 30 minutes of exclusive cluster time. When several configurations and repetitions are considered, the total cost becomes very large.
  - The parameter space explored in this work has **[TODO:** confirm final number**]** $\approx$ 7,840 configurations; measuring all of them validly with 3 repetitions would take $\approx$ 12 000 h of wall-clock time, i.e. more than one year on the target machine.

    In the parameter space considered in this work, approximately **[TODO:** ???**]** configurations have to be considered. Measuring all configurations with three repetitions would require approximately 12,000 hours of wall-clock time, which corresponds to more than one year of computation on the target machine.
  - Exhaustive tuning is therefore impossible in practice, and practitioners fall back on expert intuition or on greedy, one-parameter-at-a-time sweeps.
  - **Research question:** can a smarter search reach a near-optimal IO500 configuration in far fewer evaluations, and which search method actually does so?

    An exhaustive search is therefore not practical. Instead, practitioners often rely on their experience and knowledge of the system. Another possible approach is to change one parameter at a time and select the best value before moving to the next parameter. Such approaches can work in some cases, but they may miss interactions between parameters. A configuration that is optimal for one parameter independently may not be optimal when several parameters are considered together.

This leads to the main research question of this internship:

Can a smarter search method find a near-optimal IO500 configuration using far fewer evaluations, and which search method is most effective for this problem?

The goal is therefore not simply to find a configuration with a high IO500 score. It is also important to understand how many evaluations are required to find this configuration and how different search strategies behave under a limited evaluation budget.

- **Objectives of the internship.**
  - Build a *cheap* but faithful ground truth of the IO500 response surface on a real cluster.
  - Evaluate Bayesian Optimization (BO) as a tuning strategy for IO500, and determine *which* BO configuration works, since "Bayesian Optimization" is not a single method but a family (surrogate model × acquisition function × exploration parameter × budget).
  - Compare BO against random search, exhaustive search, and a human specialist.
  - Package the resulting methodology and configurations inside the IOPS framework, so that the study is reproducible by third parties.
- **Contributions.**
  - *A cheap ground-truth methodology.* Running the sweep with a shortened stonewall (10 s) and a small dataset produces *invalid* IO500 scores that nevertheless preserve the relative ranking of configurations. The full grid was measured in ≈ 183 h instead of more than a year.
  - *A reusable cache.* Every `parameters` → `SCORE` pair is stored in an SQLite database (`io500.db`), so any search strategy can be replayed offline at zero cluster cost with `iops run -cache-only`.
  - *A systematic comparison of BO setups*: **[TODO:** confirm**]** 83 BO configurations (surrogate ∈ {GP, RF, ET}, acquisition ∈ {EI, PI, LCB}, exploration $\xi/\kappa/n_{\text{init}}$, budget ∈ $\{10, 15, 20, 30, 40\}$), each repeated over 10 seeds.
  - *A sensitivity study* isolating which BO knob actually drives quality.
  - *A validation campaign* re-running the best configurations with a *valid* IO500 setup on the cluster, to check that the cheap proxy really does preserve the ranking.
  - *A human baseline*: a specialist challenge in which an expert submits their best guesses through a form, run under exactly the same conditions as the automated search.
- **Organisation of the report.**
  - **Context** presents the host laboratory, the team and the internship framework.
  - **Background** introduces IO500, parallel file system tuning and Bayesian Optimization.
  - **Related work** reviews existing approaches to I/O auto-tuning.

– **Tools and experimental setup** describes the tools, the target platform and the complete installation procedure.

– **Methodology** details how the ground truth was built and how the search strategies were compared.

– **Results** reports and analyses the measurements.

– **Discussion** covers the limitations and threats to validity, and **Conclusion** summarises the work and its perspectives.

Bibliography

References

[1] IO500 Foundation. The IO500 list. `https://io500.org/`, 2026. Accessed: August 13, 2026.

[2] IO500 Foundation. IO500 submission rules. `https://io500.org/rules/submission`, 2026. Accessed: August 13, 2026.

[3] TADaaM, Inria. IOPS: Integrated orchestration for parametric studies. `https://gitlab.inria.fr/hpc_io/iops`, 2026. Accessed: August 13, 2026.

[4] The scikit-optimize contributors. scikit-optimize: Sequential model-based optimization in Python. `https://scikit-optimize.github.io/`, 2026. Version TO-COMPLETE.

[5] TO-COMPLETE. To-complete: 2022 I/O performance study. `https://inria.hal.science/hal-03753813/`, 2022.

[6] TO-COMPLETE. To-complete: IO500 tuning on CLAIX23. In *IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*, 2025.

[7] TO-COMPLETE. To-complete: IO500 benchmark reference paper, 2026.